```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=yes">
  <title>CertiChain | Barangay Digital Certification System</title>
  <!-- Chart.js for analytics -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>
  <!-- jsPDF and QRCode libraries -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/qrcode@1.5.1/build/qrcode.min.js"></script>
  <!-- Firebase SDKs -->
  <script type="module">
    import { initializeApp } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-app.js";
    import { getAuth, signInWithEmailAndPassword, createUserWithEmailAndPassword, signOut,
onAuthStateChanged } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-auth.js";
    import { getFirestore, collection, addDoc, query, where, getDocs, updateDoc, doc, getDoc,
orderBy, deleteDoc } from "https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js";
    import { getStorage, ref, uploadBytes, getDownloadURL } from
"https://www.gstatic.com/firebasejs/10.7.1/firebase-storage.js";

    // FIREBASE CONFIGURATION (REPLACE WITH YOUR OWN CREDENTIALS)
    const firebaseConfig = {
      apiKey: "AIzaSyDummyKeyReplaceWithYourActualKey",  // <-- REPLACE
      authDomain: "certichain-demo.firebaseapp.com",     // <-- REPLACE
      projectId: "certichain-demo",                      // <-- REPLACE
      storageBucket: "certichain-demo.appspot.com",      // <-- REPLACE
      messagingSenderId: "123456789012",                 // <-- REPLACE
      appId: "1:123456789012:web:abcdef123456"           // <-- REPLACE
    };

    // Initialize Firebase
    const app = initializeApp(firebaseConfig);
    const auth = getAuth(app);
    const db = getFirestore(app);
    const storage = getStorage(app);

    // ----------------------- GLOBAL STATE -----------------------
    let currentUser = null;
    let currentUserRole = null;
    let trendChart = null;

    // Helper: Show/Hide views for SPA style (single file navigation)
    function showView(viewId) {
```

```
  const views = ['loginView', 'residentView', 'adminView', 'verifyView'];
  views.forEach(id => {
    const el = document.getElementById(id);
    if (el) el.style.display = id === viewId ? 'block' : 'none';
  });
}

// Logout function
async function logout() {
  await signOut(auth);
  showView('loginView');
  document.getElementById('loginEmail').value = '';
  document.getElementById('loginPassword').value = '';
}

// ----------------------- AUTH STATE HANDLER -----------------------
onAuthStateChanged(auth, async (user) => {
  if (user) {
    currentUser = user;
    const userDocRef = doc(db, "users", user.uid);
    const userSnap = await getDoc(userDocRef);
    if (userSnap.exists()) {
      currentUserRole = userSnap.data().role;
    } else {
      // fallback: if no role doc, treat as resident
      currentUserRole = "resident";
      await setDoc(userDocRef, { uid: user.uid, email: user.email, name: user.email.split('@')[0],
role: "resident" });
    }

    if (currentUserRole === "admin") {
      showView('adminView');
      loadAdminPendingRequests();
      loadAdminStatsAndChart();
    } else {
      showView('residentView');
      loadResidentRequests();
      loadDigitalVault();
    }
  } else {
    currentUser = null;
    currentUserRole = null;
    showView('loginView');
  }
```

```
});

// ----------------------- LOGIN & REGISTER (inside loginView) -----------------------
document.getElementById('showLoginBtn')?.addEventListener('click', () => {
  document.getElementById('loginFormContainer').style.display = 'block';
  document.getElementById('registerFormContainer').style.display = 'none';
});
document.getElementById('showRegisterBtn')?.addEventListener('click', () => {
  document.getElementById('loginFormContainer').style.display = 'none';
  document.getElementById('registerFormContainer').style.display = 'block';
});

// Login submission
document.getElementById('loginForm')?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const email = document.getElementById('loginEmail').value;
  const password = document.getElementById('loginPassword').value;
  try {
    const userCred = await signInWithEmailAndPassword(auth, email, password);
    // state change will handle routing
  } catch (err) {
    alert("Login failed: " + err.message);
  }
});

// Register submission
document.getElementById('registerForm')?.addEventListener('submit', async (e) => {
  e.preventDefault();
  const fullName = document.getElementById('regFullName').value;
  const email = document.getElementById('regEmail').value;
  const password = document.getElementById('regPassword').value;
  const role = document.getElementById('regRole').value;
  try {
    const userCred = await createUserWithEmailAndPassword(auth, email, password);
    const uid = userCred.user.uid;
    await setDoc(doc(db, "users", uid), {
      uid: uid,
      name: fullName,
      email: email,
      role: role
    });
    alert("Registration successful! Please login.");
    document.getElementById('loginFormContainer').style.display = 'block';
    document.getElementById('registerFormContainer').style.display = 'none';
```

```
      document.getElementById('registerForm').reset();
    } catch (err) {
      alert("Registration error: " + err.message);
    }
  });

  // Helper setDoc for Firestore
  async function setDoc(docRef, data) {
    const { setDoc } = await
import("https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore.js");
    return setDoc(docRef, data);
  }

  // ---------------------- RESIDENT DASHBOARD FUNCTIONS ----------------------
  async function loadResidentRequests() {
    if (!currentUser) return;
    const q = query(collection(db, "requests"), where("userId", "==", currentUser.uid));
    const querySnap = await getDocs(q);
    const container = document.getElementById('residentRequestsList');
    container.innerHTML = '';
    querySnap.forEach(docSnap => {
      const req = docSnap.data();
      const statusClass = req.status === 'approved' ? 'status-approved' : (req.status ===
'rejected' ? 'status-rejected' : 'status-pending');
      container.innerHTML += `
        <div class="request-card">
          <strong>${req.type}</strong><br>
          Status: <span class="${statusClass}">${req.status.toUpperCase()}</span><br>
          Requested: ${req.timestamp?.toDate?.()?.toLocaleString() || 'Just now'}
          ${req.certId ? `<br>Certificate ID: <span class="cert-id">${req.certId}</span>` : ''}
        </div>
      `;
    });
  }

  async function loadDigitalVault() {
    if (!currentUser) return;
    const q = query(collection(db, "vault"), where("userId", "==", currentUser.uid));
    const snap = await getDocs(q);
    const vaultDiv = document.getElementById('vaultDocuments');
    vaultDiv.innerHTML = '';
    if (snap.empty) {
      vaultDiv.innerHTML = '<p class="empty-state">No certificates issued yet.</p>';
      return;
```

```
    }
    snap.forEach(docSnap => {
      const cert = docSnap.data();
      vaultDiv.innerHTML += `
        <div class="vault-item">
          <div><strong>${cert.type}</strong><br><small>ID:
${cert.certId}</small><br><small>Issued: ${cert.issuedAt?.toDate?.()?.toLocaleDateString() ||
'Recent'}</small></div>
          <div><a href="${cert.pdfUrl}" target="_blank" class="btn-small">View PDF</a> <button
class="btn-small download-btn" data-url="${cert.pdfUrl}">Download</button></div>
        </div>
      `;
    });
    document.querySelectorAll('.download-btn').forEach(btn => {
      btn.addEventListener('click', (e) => {
        const url = btn.getAttribute('data-url');
        window.open(url, '_blank');
      });
    });
  }

  document.getElementById('submitRequestBtn')?.addEventListener('click', async () => {
    const certType = document.getElementById('certType').value;
    if (!currentUser) return alert("Please login");
    await addDoc(collection(db, "requests"), {
      userId: currentUser.uid,
      type: certType,
      status: "pending",
      timestamp: new Date(),
      userEmail: currentUser.email
    });
    alert("Request submitted successfully!");
    loadResidentRequests();
  });

  // Navigation tabs for resident
  document.getElementById('residentRequestTab')?.addEventListener('click', () => {
    document.getElementById('residentRequestSection').style.display = 'block';
    document.getElementById('residentVaultSection').style.display = 'none';
    loadResidentRequests();
  });
  document.getElementById('residentVaultTab')?.addEventListener('click', () => {
    document.getElementById('residentRequestSection').style.display = 'none';
    document.getElementById('residentVaultSection').style.display = 'block';
```

```
    loadDigitalVault();
  });

  // ----------------------- ADMIN DASHBOARD FUNCTIONS -----------------------
  async function loadAdminPendingRequests() {
    const q = query(collection(db, "requests"), where("status", "==", "pending"));
    const snap = await getDocs(q);
    const container = document.getElementById('adminPendingList');
    container.innerHTML = '';
    if (snap.empty) {
      container.innerHTML = '<p>No pending requests.</p>';
      return;
    }
    snap.forEach(docSnap => {
      const req = docSnap.data();
      const reqId = docSnap.id;
      container.innerHTML += `
        <div class="admin-request-card">
          <div><strong>${req.type}</strong> — ${req.userEmail ||
req.userId}<br><small>Requested: ${req.timestamp?.toDate?.()?.toLocaleString() || 'Just
now'}</small></div>
          <div>
            <button class="approve-btn" data-id="${reqId}" data-user="${req.userId}"
data-type="${req.type}">Approve</button>
            <button class="reject-btn" data-id="${reqId}">Reject</button>
          </div>
        </div>
      `;
    });
    document.querySelectorAll('.approve-btn').forEach(btn => {
      btn.addEventListener('click', () => approveRequest(btn.getAttribute('data-id'),
btn.getAttribute('data-user'), btn.getAttribute('data-type')));
    });
    document.querySelectorAll('.reject-btn').forEach(btn => {
      btn.addEventListener('click', () => rejectRequest(btn.getAttribute('data-id')));
    });
  }

  async function generateCertificatePDF(certId, userName, certType, qrDataURL) {
    const { jsPDF } = window.jspdf;
    const doc = new jsPDF();
    doc.setFontSize(20);
    doc.text("Republic of the Philippines", 20, 20);
    doc.setFontSize(16);
```

```
    doc.text("Barangay CertiChain", 20, 35);
    doc.setFontSize(12);
    doc.text(`CERTIFICATE OF ${certType.toUpperCase()}`, 20, 55);
    doc.text(`This certifies that ${userName} is a valid resident.`, 20, 70);
    doc.text(`Certificate ID: ${certId}`, 20, 85);
    doc.text(`Issued on: ${new Date().toLocaleDateString()}`, 20, 95);
    doc.text("Issued by: Barangay CertiChain System", 20, 105);
    if (qrDataURL) {
      doc.addImage(qrDataURL, 'PNG', 140, 70, 50, 50);
    }
    return doc.output('blob');
  }

  async function approveRequest(reqId, userId, certType) {
    try {
      const certId = "CERT-" + Date.now() + "-" + Math.floor(Math.random() * 10000);
      const verificationURL =
`${window.location.origin}${window.location.pathname}?verify=${certId}`;
      // Generate QR as dataURL
      const qrDataURL = await QRCode.toDataURL(verificationURL, { width: 150 });
      // Get user name
      let userName = "Resident";
      const userDoc = await getDoc(doc(db, "users", userId));
      if (userDoc.exists()) userName = userDoc.data().name || userDoc.data().email;
      // Create PDF
      const pdfBlob = await generateCertificatePDF(certId, userName, certType, qrDataURL);
      const storagePath = `certificates/${certId}.pdf`;
      const storageRef = ref(storage, storagePath);
      await uploadBytes(storageRef, pdfBlob);
      const downloadURL = await getDownloadURL(storageRef);
      // Update request status
      await updateDoc(doc(db, "requests", reqId), {
        status: "approved",
        certId: certId,
        pdfUrl: downloadURL,
        qrCode: verificationURL,
        approvedAt: new Date()
      });
      // Add to digital vault
      await addDoc(collection(db, "vault"), {
        userId: userId,
        certId: certId,
        type: certType,
        pdfUrl: downloadURL,
```

```
      qrCode: verificationURL,
      issuedAt: new Date()
    });
    alert("Certificate approved and stored.");
    loadAdminPendingRequests();
    loadAdminStatsAndChart();
  } catch (err) {
    console.error(err);
    alert("Approval failed: " + err.message);
  }
}

async function rejectRequest(reqId) {
  await updateDoc(doc(db, "requests", reqId), { status: "rejected", rejectedAt: new Date() });
  alert("Request rejected.");
  loadAdminPendingRequests();
  loadAdminStatsAndChart();
}

async function loadAdminStatsAndChart() {
  const allRequestsSnap = await getDocs(collection(db, "requests"));
  const total = allRequestsSnap.size;
  let approved = 0, rejected = 0, pending = 0;
  allRequestsSnap.forEach(d => {
    const st = d.data().status;
    if (st === 'approved') approved++;
    else if (st === 'rejected') rejected++;
    else pending++;
  });
  document.getElementById('totalRequestsCount').innerText = total;
  document.getElementById('approvedCount').innerText = approved;
  document.getElementById('rejectedCount').innerText = rejected;
  const ctx = document.getElementById('trendChartCanvas').getContext('2d');
  if (trendChart) trendChart.destroy();
  trendChart = new Chart(ctx, {
    type: 'bar',
    data: {
      labels: ['Approved', 'Rejected', 'Pending'],
      datasets: [{
        label: 'Requests',
        data: [approved, rejected, pending],
        backgroundColor: ['#2c7da0', '#a4161a', '#ffb703'],
        borderRadius: 8
      }]
```

```
      },
      options: { responsive: true, maintainAspectRatio: true }
    });
  }

  // ---------------------- PUBLIC VERIFICATION PAGE ----------------------
  async function verifyCertificateById(certId) {
    const vaultQuery = query(collection(db, "vault"), where("certId", "==", certId));
    const snap = await getDocs(vaultQuery);
    const resultDiv = document.getElementById('verificationResult');
    if (snap.empty) {
      resultDiv.innerHTML = '<div class="verify-invalid"><span>❌ INVALID
CERTIFICATE</span><br>No matching record found.</div>';
      return;
    }
    const certData = snap.docs[0].data();
    // get user name
    let userName = "Resident";
    const userDoc = await getDoc(doc(db, "users", certData.userId));
    if (userDoc.exists()) userName = userDoc.data().name || userDoc.data().email;
    resultDiv.innerHTML = `
      <div class="verify-valid">
        <span>✅ VALID CERTIFICATE</span>
        <p><strong>Certificate ID:</strong> ${certData.certId}</p>
        <p><strong>Issued to:</strong> ${userName}</p>
        <p><strong>Certificate Type:</strong> ${certData.type}</p>
        <p><strong>Issuing Barangay:</strong> Barangay CertiChain</p>
        <p><strong>Status:</strong> Active / Valid</p>
        <p><strong>Issue Date:</strong> ${certData.issuedAt?.toDate?.()?.toLocaleDateString()
|| 'Valid'}</p>
        <a href="${certData.pdfUrl}" target="_blank" class="btn-small">Download Official
PDF</a>
      </div>
    `;
  }

  // Verify page logic
  document.getElementById('verifyBtn')?.addEventListener('click', async () => {
    const input = document.getElementById('certIdInput').value.trim();
    if (!input) return alert("Enter certificate ID or scan QR content.");
    await verifyCertificateById(input);
  });

  // Optional: parse URL param for direct verify link
```

```
  const urlParams = new URLSearchParams(window.location.search);
  const verifyParam = urlParams.get('verify');
  if (verifyParam && document.getElementById('verifyView') &&
document.getElementById('verifyView').style.display !== 'none') {
    document.getElementById('certIdInput').value = verifyParam;
    verifyCertificateById(verifyParam);
  }

  // ----------------------- GLOBAL UI NAVIGATION (Single File Routing) -----------------------
  // Navigation buttons for all views
  document.getElementById('navHome')?.addEventListener('click', (e) => {
    e.preventDefault();
    if (currentUserRole === 'admin') showView('adminView');
    else if (currentUser) showView('residentView');
    else showView('loginView');
  });
  document.getElementById('navRequest')?.addEventListener('click', (e) => {
    e.preventDefault();
    if (currentUserRole === 'admin') alert("Admin use dashboard.");
    else {
      showView('residentView');
      document.getElementById('residentRequestSection').style.display = 'block';
      document.getElementById('residentVaultSection').style.display = 'none';
      loadResidentRequests();
    }
  });
  document.getElementById('navCertificates')?.addEventListener('click', (e) => {
    e.preventDefault();
    if (currentUserRole === 'admin') alert("Admin vault not applicable.");
    else {
      showView('residentView');
      document.getElementById('residentRequestSection').style.display = 'none';
      document.getElementById('residentVaultSection').style.display = 'block';
      loadDigitalVault();
    }
  });
  document.getElementById('navVerify')?.addEventListener('click', (e) => {
    e.preventDefault();
    showView('verifyView');
  });
  document.getElementById('navDashboard')?.addEventListener('click', (e) => {
    e.preventDefault();
    if (currentUserRole === 'admin') showView('adminView');
    else if (currentUser) showView('residentView');
```

```
    else showView('loginView');
  });
  document.getElementById('logoutBtnGlobal')?.addEventListener('click', logout);
  document.getElementById('adminLogoutBtn')?.addEventListener('click', logout);
  document.getElementById('verifyLogoutBtn')?.addEventListener('click', logout);

  // Initial load: hide all views first, show login
  showView('loginView');

  // Additional UI helpers: load admin data when view becomes active
  const originalShowView = showView;
  window.showView = function(viewId) {
    originalShowView(viewId);
    if (viewId === 'adminView' && currentUserRole === 'admin') {
      loadAdminPendingRequests();
      loadAdminStatsAndChart();
    }
    if (viewId === 'residentView' && currentUser) {
      loadResidentRequests();
      if (document.getElementById('residentRequestSection').style.display !== 'none')
loadResidentRequests();
    }
  };
  showView = window.showView;
 </script>

 <style>
  * { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Inter', system-ui, -apple-system,
'Segoe UI', Roboto; }
  body { background: #f0f4f8; }
  .navbar { background: #0b2b40; color: white; padding: 1rem 2rem; display: flex; flex-wrap:
wrap; justify-content: space-between; align-items: center; gap: 1rem; }
  .nav-links { display: flex; gap: 1.5rem; flex-wrap: wrap; }
  .nav-links a, .nav-links button { color: white; text-decoration: none; background: none; border:
none; font-weight: 500; cursor: pointer; font-size: 1rem; }
  .nav-links a:hover, button:hover { opacity: 0.8; }
  .btn-logout { background: #b23c1c; padding: 0.3rem 1rem; border-radius: 2rem; }
  .container { max-width: 1300px; margin: 2rem auto; padding: 0 1.5rem; }
  .glass-card, .dashboard-card { background: white; border-radius: 2rem; padding: 2rem;
box-shadow: 0 8px 20px rgba(0,0,0,0.05); margin-bottom: 2rem; border: 1px solid #e2e8f0; }
  h1, h2, h3 { color: #0b2b40; margin-bottom: 1rem; }
  .toggle-buttons { display: flex; gap: 1rem; margin-bottom: 1.5rem; }
  .toggle-buttons button { background: #e2e8f0; color: #0b2b40; padding: 0.5rem 1.5rem;
border-radius: 2rem; }
```

```
    .toggle-buttons button.active { background: #2c7da0; color: white; }
    input, select { width: 100%; padding: 0.8rem; margin: 0.5rem 0 1rem; border-radius: 1rem;
border: 1px solid #cbd5e1; background: #fefefe; }
    button { background: #2c7da0; color: white; border: none; padding: 0.6rem 1.5rem;
border-radius: 2rem; cursor: pointer; font-weight: 600; transition: 0.2s; }
    button:hover { background: #1f5068; }
    .request-card, .admin-request-card, .vault-item { background: #f9fafc; border-radius: 1.2rem;
padding: 1rem; margin-bottom: 1rem; border-left: 6px solid #2c7da0; display: flex;
justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 0.8rem; }
    .admin-request-card { border-left-color: #e67e22; }
    .status-approved { color: #15803d; font-weight: bold; }
    .status-rejected { color: #b91c1c; }
    .status-pending { color: #d97706; }
    .stats-grid { display: flex; gap: 1.5rem; flex-wrap: wrap; margin-bottom: 2rem; }
    .stat-card { background: white; border-radius: 1.5rem; padding: 1.2rem; flex: 1; text-align:
center; box-shadow: 0 2px 8px rgba(0,0,0,0.05); font-size: 1.4rem; font-weight: bold;
border-bottom: 4px solid #2c7da0; }
    .btn-small { background: #2c7da0; padding: 0.3rem 0.8rem; font-size: 0.8rem; border-radius:
2rem; text-decoration: none; color: white; display: inline-block; }
    .vault-item { justify-content: space-between; }
    .verify-container { max-width: 600px; margin: 2rem auto; }
    .verify-valid { background: #dcfce7; padding: 1.5rem; border-radius: 1.5rem; border-left: 8px
solid #15803d; margin-top: 1rem; }
    .verify-invalid { background: #fee2e2; padding: 1.5rem; border-radius: 1.5rem; border-left: 8px
solid #b91c1c; }
    .empty-state { color: #64748b; text-align: center; padding: 2rem; }
    canvas { max-height: 300px; margin-top: 1rem; }
    @media (max-width: 700px) { .navbar { flex-direction: column; align-items: flex-start; }
.stats-grid { flex-direction: column; } .request-card { flex-direction: column; align-items: flex-start;
} }
    .section-title { margin-top: 1rem; }
  </style>
</head>
<body>

<!-- LOGIN VIEW -->
<div id="loginView" style="display:none">
  <div class="navbar"><h2>🏛️ CertiChain</h2><div class="nav-links"><a href="#"
id="navHome">Home</a><a href="#" id="navVerify">Verify</a></div></div>
  <div class="container">
    <div class="glass-card">
      <h1>Barangay Certification System</h1>
      <div class="toggle-buttons"><button id="showLoginBtn"
class="active">Login</button><button id="showRegisterBtn">Register</button></div>
```

```
    <div id="loginFormContainer"><form id="loginForm"><input type="email" id="loginEmail"
placeholder="Email" required><input type="password" id="loginPassword"
placeholder="Password" required><button type="submit">Login</button></form></div>
    <div id="registerFormContainer" style="display:none"><form id="registerForm"><input
type="text" id="regFullName" placeholder="Full Name" required><input type="email"
id="regEmail" placeholder="Email" required><input type="password" id="regPassword"
placeholder="Password" required><select id="regRole"><option
value="resident">Resident</option><option value="admin">Barangay Staff
(Admin)</option></select><button type="submit">Register</button></form></div>
   </div>
  </div>
</div>

<!-- RESIDENT VIEW -->
<div id="residentView" style="display:none">
  <div class="navbar"><h2>📄 CertiChain | Resident Portal</h2><div class="nav-links"><a
href="#" id="navHome">Home</a><a href="#" id="navRequest">Request</a><a href="#"
id="navCertificates">Digital Vault</a><a href="#" id="navVerify">Verify</a><button
id="logoutBtnGlobal" class="btn-logout">Logout</button></div></div>
  <div class="container">
   <div class="dashboard-card"><div style="display: flex; gap: 1rem; margin-bottom:
1rem;"><button id="residentRequestTab">📋 My Requests</button><button
id="residentVaultTab">📁 Digital Vault</button></div>
    <div id="residentRequestSection"><h3>Request New Certificate</h3><select
id="certType"><option value="Barangay Clearance">Barangay Clearance</option><option
value="Residency">Certificate of Residency</option><option value="Indigency">Certificate of
Indigency</option></select><button id="submitRequestBtn">Submit Request</button><h3
class="section-title">My Request History</h3><div id="residentRequestsList"></div></div>
    <div id="residentVaultSection" style="display:none"><h3>🔐 Your Digital Vault</h3><div
id="vaultDocuments"></div></div>
   </div>
  </div>
</div>

<!-- ADMIN VIEW -->
<div id="adminView" style="display:none">
  <div class="navbar"><h2>⚙️ CertiChain Admin Dashboard</h2><div class="nav-links"><a
href="#" id="navDashboard">Analytics</a><a href="#" id="navVerify">Verify</a><button
id="adminLogoutBtn" class="btn-logout">Logout</button></div></div>
  <div class="container">
   <div class="stats-grid"><div class="stat-card">📊 Total Requests: <span
id="totalRequestsCount">0</span></div><div class="stat-card">✅ Approved: <span
id="approvedCount">0</span></div><div class="stat-card">❌ Rejected: <span
id="rejectedCount">0</span></div></div>
```

```
    <div class="dashboard-card"><h3>📈 Request Trends</h3><canvas id="trendChartCanvas"
width="400" height="200"></canvas></div>
    <div class="dashboard-card"><h3>🕒 Pending Requests for Action</h3><div
id="adminPendingList"></div></div>
  </div>
</div>

<!-- PUBLIC VERIFY VIEW -->
<div id="verifyView" style="display:none">
  <div class="navbar"><h2>🔍 CertiChain Public Verification</h2><div class="nav-links"><a
href="#" id="navHome">Home</a><button id="verifyLogoutBtn">Logout</button></div></div>
  <div class="container verify-container">
    <div class="glass-card"><h3>Verify Certificate Authenticity</h3><input type="text"
id="certIdInput" placeholder="Enter Certificate ID or scan QR code"><button
id="verifyBtn">Verify Now</button><div id="verificationResult" style="margin-top:
1.5rem;"></div></div>
  </div>
</div>

<script>(function() { console.log("CertiChain ready — replace firebase config to enable full
backend"); })();</script>
</body>
</html>
```